UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

DEPARTAMENTO ACADÉMICO DE INGENIERÍA INFORMÁTICA

REDES DE COMPUTADORAS 2

Lab 03: Sockets e Hilos (TCP) en Python

Leunam

"Dime y lo olvidaré. Muéstrame y lo recordaré. Involúcrame y lo entenderé"

Proverbio chino

---

1. OBJETIVOS:

1.1. Entender lo básico de la comunicación por redes.
1.2. Programar aplicaciones de red usando sockets.
1.3. Aprender cómo construir aplicaciones cliente/servidor que se comunican usando sockets
1.4. Aprender el uso de sockets en Python.
1.5. Aprender el uso de hilos y excepciones en Python.

2. BASE TEORICA COMPLENTARIA:

Socket: Una interface creada, controlada por la aplicación del host local y controlada por el SO, en el cual el proceso aplicación puede tanto enviar como recibir mensajes a ó desde otro proceso aplicación.

3. RECURSOS:

3.1. Python.
3.2. Ubuntu/Windows.

4. DESARROLLO DE LA PRACTICA:

En el presente trabajo de laboratorio se implementará en TCP el servicio concurrente CAPITALIZAR UN STRING, es decir uno o varios clientes envían cadenas de caracteres a un servidor, el servidor recibe estos strings desde los clientes, los capitaliza y devuelve cada string capitalizado de regreso al respectivo cliente.

4.1. Escribir una aplicación Cliente-Servidor que implemente el servicio "CAPITALIZAR UN STRING". que pueda atender múltiples peticiones.

APLICACIÓN SERVIDOR:
ServidorTCP_MC.py

```
# ServidorTCP_MC.py

import socket
import threading
import HiloClienteTCP

# inicializar variables
idServidor = ''                 # nombre simbólico que significa todas las interfaces disponibles
puertoServidor = 12000

# crear socket de escucha (o de bienvenida)
#       1er. parámetro: familia de direcciones: AF_INET indica IPv4
#       2do. parámetro: tipo de socket: SOCK_STREAM significa socket TCP
socketServidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Set opciones del socket
#       categoría de opciones: SOL_SOCKET
#       opción/flag a usar: SO_REUSEADDR --> dice al kernel reusar un socket local en
#       estado TIME_WAIT, sin esperar por su timeout natural a expirar
socketServidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
# Enlazar el socket al id_Servidor y al nro. puerto del Servidor
socketServidor.bind((idServidor, puertoServidor))

# Servidor se queda escuchando por peticiones de conexión TCP
#       (desde el cliente, con un máximo de conexiones en cola de 1)
socketServidor.listen(1)

print ('El servidor está listo para escuchar')

# Bucle 'eterno'
while True:
        # El servidor espera en el método accept() por peticiones entrantes,
        #   un nuevo socket hijo dedicado para este cliente en particular es
        #   creado al retorno de este método ==> socketConexion
        (socketConexion, (ipCliente, puertoCliente)) = socketServidor.accept()

        # ----

        # crear hilo
        hilo = HiloClienteTCP.HiloCliente(ipCliente, puertoCliente, socketConexion)
        hilo.start()

# cerrar socket servidor (nunca debería ocurrir)
socketServidor.close()
```

HiloClienteTCP.py

```
# HiloClienteTCP.py
import socket
import threading

class HiloCliente (threading.Thread):
        # Constructor
        def __init__(self, ip, puerto, socket):
        threading.Thread.__init__(self)

        self.nroIP = ip
        self.nroPuerto = puerto
        self.socketCnx = socket


        def run(self):

        mensaje = 'xxxx'          # solo para el propósito de entrar al bucle while

        while len(mensaje):
                # Leer solo los bytes desde el socket
                #mensaje = self.socketCnx.recv(1024)
                mensaje = self.socketCnx.recv(1024).decode()

                # Capitalizar el mensaje
                mensajeCapitalizado = mensaje.upper()

                # Enviar mensaje capitalizado de regreso al cliente
                #self.socketCnx.send(mensajeCapitalizado)
                self.socketCnx.send(mensajeCapitalizado.encode())

                # ----

                # ----
```

```
        # Cerrar socket de conexión en el hilo
        self.socketCnx.close()
```

APLICACIÓN CLIENTE:

ClienteTCP_MC.py

```
# ClienteTCP_MC.py

from socket import *

# Inicializar variables
idServidor = 'dirección_IP'
puertoServidor = 12000

# Crear socket TCP para comunicarse con el servidor
socketCliente = socket(AF_INET, SOCK_STREAM)

# Iniciar conexión TCP hacia el servidor, indicando la dirección del servidor,
#       es decir el id_Servidor y el puerto remoto 12000
socketCliente.connect((idServidor, puertoServidor))

mensaje = ''

# Obtener alguna entrada desde el teclado del usuario
mensaje = input('Entrar un mensaje en minúsculas: ')

while mensaje != "":
        # enviar el string a través del socket del cliente hacia la conexión TCP
        socketCliente.send(mensaje.encode())

        # ¡El cliente se queda en espera!

        # leer desde su socket, los datos son colocados en el string mensajeModificado
        mensajeModificado = socketCliente.recv(1024)

        # imprimir string recibido
        print ('Respuesta del servidor:', mensajeModificado.decode())

        # ----

        # ----

        # obtener alguna entrada desde el teclado del usuario
        mensaje = input('Entrar un mensaje en minúsculas:')

# Cerrar socket
socketCliente.close()
```

4.2. Agregar código en la aplicación servidor para que muestre en cada petición/respuesta la información de la conexión TCP y la información del hilo, es decir en el extremo del host servidor se tiene que visualizar la información del socket cliente y la del socket servidor, y el hilo en el que es atendido cada uno de los clientes.

4.3. Agregar código en la aplicación cliente para que muestre en cada petición/respuesta la información de la conexión TCP, es decir en el extremo del host cliente se tiene que visualizar la información del socket cliente y la del socket servidor.

4.4. En 3 equipos diferentes abrir 1 ventana de CLI (Interface de Línea de Comandos) o de Terminal, y en la ventana cambiarse al directorio donde están los archivos Python.

ServidorTCP_MC.py
HiloClienteTCP.py
ClienteTCP_MC.py

- En la ventana del servidor ejecutar el programa Servidor:

  python `ServidorTCP_MC.py`
  o
  py `ServidorTCP_MC.py`
  o
  python3 `ServidorTCP_MC.py`

  Nota: El archivo Python `HiloClienteTCP.py` *debe estar en el mismo directorio del Servidor.*

- En las otras 2 ventanas del cliente ejecutar el programa Cliente:

  python `ClienteTCP_MC.py`
  o
  py `ClienteTCP_MC.py`
  o
  python3 `ClienteTCP_MC.py`

  (tipear un texto en minúsculas y presionar Enter)

5. EJERCICIOS PROPUESTOS:

5.1. Averiguar el valor de la dirección IP Pública de la UNSAAC, i.e., la dirección IP con la que la Red Institucional es vista desde el Internet.

Tipo de trabajo	Individual
Tipo de informe	Digital: PDF
Fecha de entrega	dd/mm/aaaa